

IX: Techniques de Deep learning et réseaux à convolution

Guillaume Guex guillaume.guex@unil.ch

Techniques de Deep learning

On parle de **deep learning** (apprentissage profond) lorsque l'algorithme utilisé possède **plusieurs couches de représentation** qui enchainent **les degrés d'abstraction dans la représentation des données**.

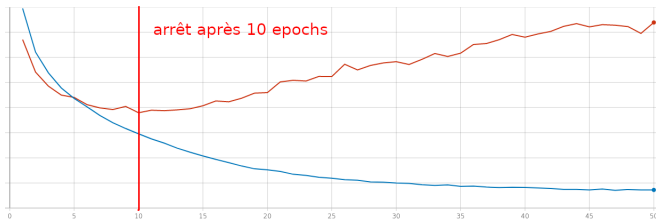
Les algorithmes de deep learning sont très puissants, et peuvent **apprendre des fonctions très complexes** sur les données.

Cependant, ces algorithmes nécessitent de **nombreuses données**, sont **gourmands en ressources computationnelles** et peuvent être sujet à la **sur-paramétrisation**.

Différentes techniques permettent de réduire ces problèmes.

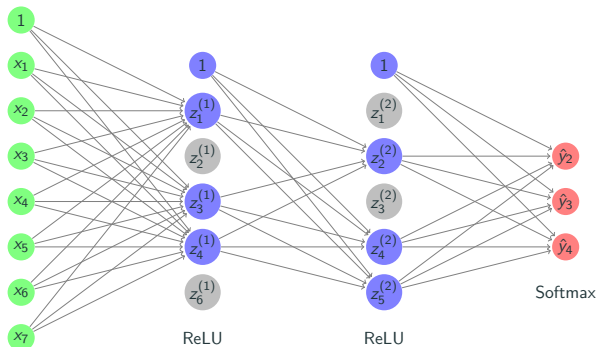
L'arrêt prématuré

Une manière de **régulariser** un réseau de neurones est d'utiliser un **jeu de validation** pour suivre la perte de généralisation. On peut alors arrêter l'entraînement lorsque cette perte stagne ou augmente à nouveau. C'est l'**arrêt prématuré**.



Le dropout

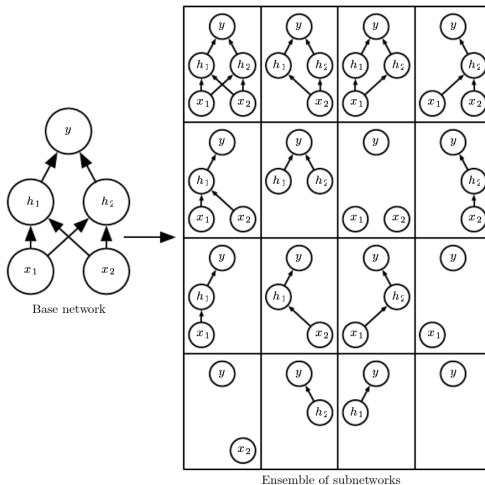
Une autre manière de régulariser un réseau de neurones et d'utiliser le **dropout**. Cette technique consiste à **désactiver aléatoirement certains neurones** (leur résultat est mis à zéro) à chaque itération de l'entraînement du réseau.



Lors de l'évaluation, on réactive tous les neurones.

Le dropout

On évite ainsi de sur-spécialiser les neurones. Le dropout peut être vu comme une forme de **méthode d'ensemble**.



Augmentation des données

Une autre forme de régularisation est d'effectuer une **augmentation des données** en modifiant légèrement les données d'origine. Elle est particulièrement utilisée avec les **images**.



Source : <https://deeplylearning.fr/cours-theoriques-intelligence-artificielle/data-augmentation/>

Une technique qui évite un entraînement complet d'un modèle est d'effectuer des **réglages fins (fine tuning)** sur un **modèle pré-entraîné**. Cela consiste à reprendre un modèle déjà entraîné sur des données similaires, puis de continuer l'entraînement avec notre propre jeu de données.

C'est une technique appartient au sous-domaine de l'**apprentissage par transfert (transfer learning)**.

La normalisation par batch

La **normalisation par batch** est une technique qui a nombreux bénéfices sur l'entraînement d'un réseau de neurones. L'idée est de prendre les résultats y_{ij} (individu i sur neurone j) d'un batch $\mathcal{B}$ sur une couche **avant la fonction d'activation**. On va alors normaliser les résultats de chaque neurone :

$$y_{ij}^{\text{norm}} = \frac{y_{ij} - \mu_j}{\sigma_j + \epsilon}$$
$$\mu_j = \frac{1}{n_{\mathcal{B}}} \sum_{i \in \mathcal{B}} y_{ij} \quad \sigma_j^2 = \frac{1}{n_{\mathcal{B}}} \sum_{i \in \mathcal{B}} (y_{ij} - \mu_j)^2$$

Qui sont ensuite remis à l'échelle avec des **paramètres entraînables** γ_j et β_j :

$$y_{ij}^{\text{new}} = \gamma_j y_{ij}^{\text{norm}} + \beta_j$$

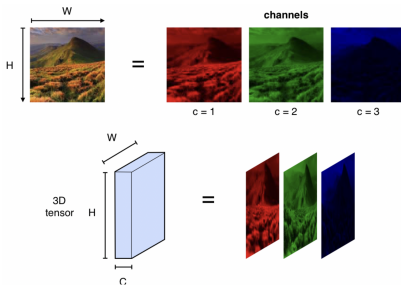
La nouvelle valeur y_{ij}^{new} passe ensuite la fonction d'activation.

Réseaux à convolution

Les réseaux à convolution

Les **réseau de neurones à convolution (CNN)** sont des réseaux qui utilisent une architecture particulière inspirée de notre **cortex visuel**. Ils sont particulièrement utilisés pour le traitement des **images**.

Une image en couleur avec $(h \times w)$ pixels peut être vu comme un **tenseur** de forme $(3 \times h \times w)$. Les couleurs sont sur des **canaux**.

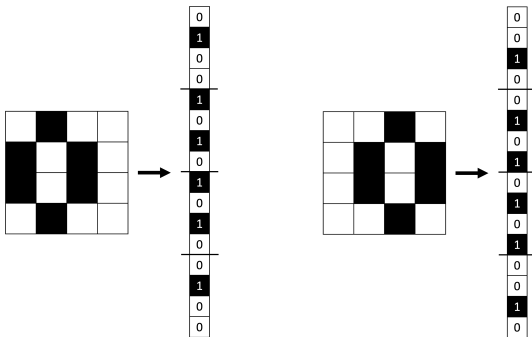


Perceptron multicouche pour les images

En transformant ce tenseur en vecteur, on pourrait essayer de classifier des images avec un **perceptron multicouche**.

Seulement :

- Le **nombre de paramètres est très élevé**.
- Les vecteurs d'entrée sont **sensibles aux translations**.



Les **réseaux à convolution** utilisent trois idées pour résoudre ces problèmes :

- Des **connections creuses** (sparse connections).
- Le **partage des poids**.
- Une **invariance à la translation**.

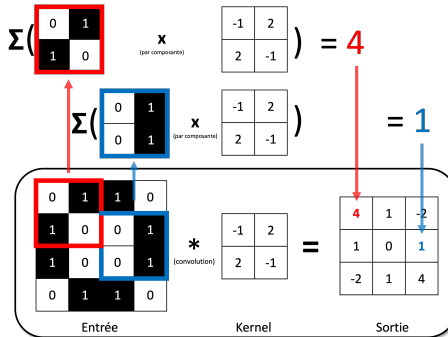
Ces idées sont mises en œuvre à travers deux couches de neurones spécifiques aux réseaux à convolution :

- Les **couches de convolution**.
- Les **couches de pooling**.

Les couches de convolution

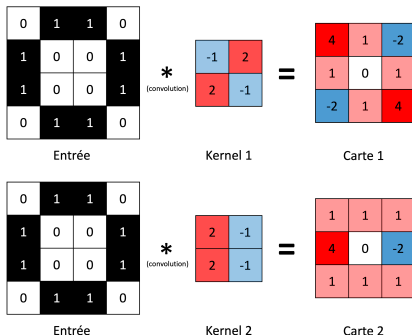
Une **couche de convolution** est constituée en plusieurs **kernels** $\mathbf{K}^m = (k_{ab}^m)$, de taille $d \times d$, qui vont effectuer une opération de **convolution** sur les données $\mathbf{X} = (x_{ij})$. Les noyaux sont **entraînés**.

$$y_{ij}^m = (\mathbf{X} * \mathbf{K}^m)_{ij} = \sum_{a=1}^d \sum_{b=1}^d x_{i+a, j+b} k_{ab}^m$$



Les kernels

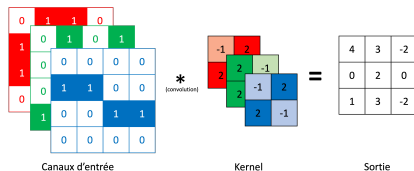
Un **kernel** (aussi appelé filtre ou calque), une fois entraîné, va détecter l'**adéquation entre les entrées et une certaine disposition de pixels (une forme) partout sur l'image.**



Avec m kernels, on obtient **m cartes de sortie**, autrement dit **m nouveaux canaux.**

Les kernels

En réalité, chaque kernel fait une convolution sur les **c canaux précédents**, et est donc un tenseur de taille $(c \times d \times d)$.

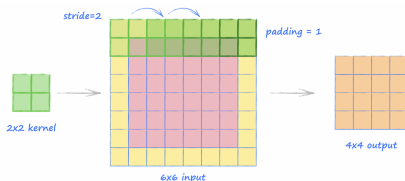


Le **nombre de canaux**, leur **hauteur** et leur **largeur** changent après une couche de convolution.

Le stride et le padding

Le nombre de canaux de sortie correspond aux nombres m de kernels utilisés. La dimension des cartes de sorties dépend :

- De **la taille du kernel** $d \times d$.
- Du **stride** s : le décalage entre les positions du kernel.
- Du **padding** p : le débordement permis au kernel.



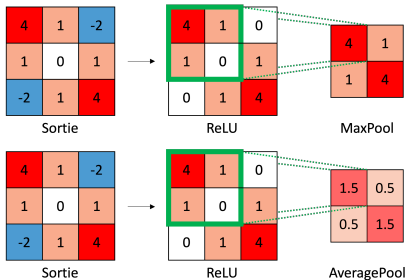
Source : C. Yanagisawa (2021) Conv2d and ConvTransposed2d

Pour une largeur d'entrée w_{in} , la largeur de sortie sera :

$$w_{out} = \left\lfloor \frac{w_{in} + 2p - d}{s} + 1 \right\rfloor$$

Le pooling

Après une couche de convolution et sa fonction d'activation (souvent ReLU), on trouve souvent une couche de **pooling**. Cette couche **résume l'information** contenue dans une grille $d \times d$ en un seul nombre.



En pratique, le **MaxPool** donne fréquemment de meilleurs résultats.

Il est possible définir un **stride** et un **padding** particulier pour les couches de pooling, avec une modification de la taille de la grille identique à la convolution.

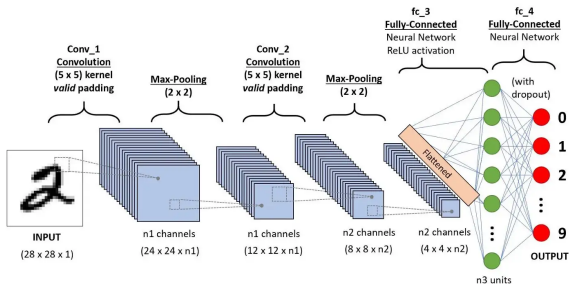
Le pooling a les bénéfices suivants :

- Il rend le réseau résistant **aux petits translations locales**.
- Il **réduit le nombre de features**, et donc de le nombre de paramètres à entraîner dans la couche suivante.
- Il introduit une forme de **régularisation**.

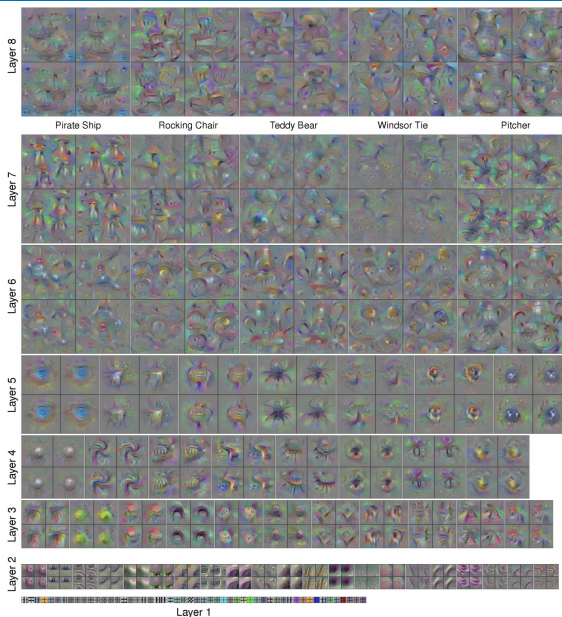
Un exemple d'architecture

Dans une **architecture typique** d'un réseau à convolution :

- La première partie du réseau est constituée de la **séquence de couches convolutions-activation-pooling**.
- Le résultat intermédiaire est **aplati (vectorisé)**.
- La dernière partie est un **réseau entièrement connecté**, se chargeant de la tâche en question.

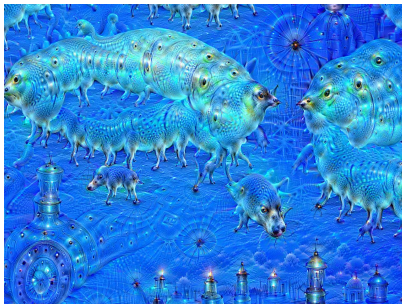


Observer un réseau à convolution entraîné



Source : J. Yosinski et al. (2015) *Understanding Neural Networks Through Deep Visualization*

Utilisation détournée : DeepDream



Source : <https://en.wikipedia.org/wiki/DeepDream>